Alerts and print sheets

qp's live signal watcher, and the sheets that print a register day (or any list of stocks) as charts and spreadsheets. Written from `quant-platform/chart/alerts.py` and the print routes in `chart/server.py`.

Related: the strategies the watcher evaluates (STRATEGIES.md), the registers it reads (REGISTERS.md).

The signal watcher (🔔 Alerts)

Watches today's R1 and Shortlist across the screener tools, evaluates every saved strategy on live 1-minute bars, and raises an alert when a strategy's entry is true on the last completed bar. The page turns each alert into a sound and a browser notification. It alerts only: nothing is ordered.

Design point	How
Symbols	the union of today's R1 and Shortlist registers, re-read every cycle, so a stock flagged at 11:00 is watched from the next cycle
Strategies	every saved strategy, re-read every cycle: a strategy saved next week is watched automatically
Session	cycles run **09:25–16:05 ET, Monday to Friday**. A strategy outside its own window (`window_start`–`window_end`, with 5 minutes of grace before it) is skipped
Signal	`evaluate(…, days=1, fill='close')`: the entry group is true on the last completed bar
Feed	the desk's default feed, read at every start (it used to be fixed to Polygon, which is a day behind on the free plan)
Interval	every 60 seconds by default (at least 15)
No repeats	one alert per strategy, symbol and signal bar; while a signal stays true, repeats are held back for **10 minutes**. In memory only: a restart may alert once more
Isolation	one bad symbol or strategy never stops a cycle; errors are kept and shown in the status
Cost	no shared bar cache across the cycle: each (strategy, symbol) pair is one fetch per cycle

Route	Does
`POST /api/alerts/start`	start (idempotent); `interval` in seconds
`POST /api/alerts/stop`	stop
`GET /api/alerts/status`	on or off, what it watches, last cycle, errors
`GET /api/alerts/recent?since_ts=`	alerts newer than a time; the page polls this

The watcher evaluates with the `close` fill, while backtests default to `desk`: an alert can fire on a different bar from the one a backtest of the same strategy would trade (Q-N9).

Print sheets

For reviewing what the scanners found the way a trader reviews charts: one chart per stock, printable, with the numbers behind it.

Route	Gives
`GET /api/r1/print`	**the day-trade sheet**: for every register day in a range (`start`/`end` or `day`), one chart per ticker over **full extended hours (04:00–20:00 ET)**, from `days_before` trading days before (default 1) to `days_after` after (default 0); your timeframe (default 1m), feed and indicators (`overlays`); `cols` per row. Stops at **150 charts** and says so
`GET /api/r1/swing`	**the swing sheet**: one **daily** chart per ticker, a year (`lookback`, 252 sessions) before the day the scanner found it, that day marked, and every session since
`GET /api/r1/csv`	the day-trade sheet's numbers: one row per bar, with register day, symbol, time, session (pre / rth / post), OHLCV and **one column per indicator**, named as the sheet's legend names it; warm-up left empty, never zero
`GET /api/r1/swing.csv`	the swing sheet's numbers, plus `phase` (before / found / after) and `pct_from_found`
`GET /api/r1/cards.csv`	every column the screener froze for each ticker that morning (score, RVOL, gap, ATR, regime, sector bias, catalyst …): why it was on the list; no bar data needed
`POST /api/pairs/parse`	shows how a pasted list will be read, before anything is fetched
`GET /api/pairs/print`, `/api/pairs/csv`	the same sheet and numbers for an explicit list of `symbol, date` pairs (a journal or review list: one per line, a tab-separated table with extra columns, or that table pasted vertically), optionally with trades drawn

All sheets take `register` (a tool’s R1 or Shortlist, or all tools merged) and are self-contained HTML pages, so the browser’s print works.